

Stima della cardinalità nei flussi di dati distribuiti: merge e compatibilità semantica degli algoritmi di sketch

Seminario di tesi magistrale

Daniele Ferroli

Università degli Studi di Udine

A.A. 2024–2025

- ① Motivazione, domanda di ricerca e contributi
- ② Background essenziale sugli sketch count-distinct
- ③ Disegno sperimentale e infrastruttura
- ④ Risultati sul comportamento in streaming
- ⑤ Risultati sul merge distribuito
- ⑥ Conclusioni, limiti e sviluppi futuri

Problema e motivazione

- Il problema riguarda la stima degli **elementi distinti** che compaiono in un flusso di dati continuo.
- Il conteggio esatto diventa rapidamente costoso in memoria e tempo quando il numero di elementi distinti cresce e la risposta deve essere aggiornata con bassa latenza.
- Le applicazioni tipiche includono telemetria, monitoraggio di rete, osservabilità, analisi di log e sistemi distribuiti.
- Nei contesti distribuiti non basta una buona stima locale: serve anche poter **combinare correttamente** sketch prodotti su nodi diversi.

Punto chiave

Il tema del lavoro non è solo quanto bene uno sketch stimi F_0 , ma cosa accade quando il **merge** viene fatto in situazioni eterogenee o non ideali.

Dal conteggio esatto allo sketch

Conteggio esatto

$$S_t = \langle x_1, \dots, x_t \rangle, \quad V_t = \{x_j \mid 1 \leq j \leq t\}$$

$$V_i \leftarrow V_{i-1} \cup \{x_i\}, \quad |V_t|$$

Memoria $\Theta(|V_t|) = \Theta(F_0(t))$: cresce con il numero di distinti osservati.

Sketch

$$S_t = \langle x_1, \dots, x_t \rangle, \quad M_0 = \text{sketch vuota}$$

$$M_i \leftarrow \text{Update}(M_{i-1}, x_i)$$

$$\widehat{F_0}(t) \leftarrow \text{Query}(M_t)$$

Si mantiene uno stato compatto e si restituisce una stima di $F_0(t)$.

- L'hash trasforma gli elementi in segnali pseudo-casuali osservabili nello sketch.
- Il compromesso fondamentale è tra memoria, costo di aggiornamento e accuratezza della stima.
- In un contesto distribuito c'è un ulteriore requisito: il merge degli sketch locali.

Caso distribuito e domanda di ricerca

$$S_1 \longrightarrow K_1 \quad S_2 \longrightarrow K_2 \quad S_3 \longrightarrow K_3$$

Per un algoritmo fissato $\mathcal{A}$, $\oplus_{\mathcal{A}}$ denota il merge su sketch compatibili; K_{serial} è lo sketch ottenuto processando $S_1 \parallel S_2 \parallel S_3$, con $\parallel$ concatenazione dei flussi.

$$\text{Query}(K_1 \oplus_{\mathcal{A}} K_2 \oplus_{\mathcal{A}} K_3) = \text{Query}(K_{\text{serial}})$$

- Ogni nodo mantiene uno sketch locale del proprio sottoflusso o della propria partizione temporale.
- In un sistema possono cambiare algoritmo, funzione di hash, seed e precisione dello sketch.
- Il punto è capire quando il merge coincide ancora col caso seriale.

Domanda di ricerca

Quando il merge è semanticamente corretto, quando è recuperabile e quando invece deve essere rifiutato?

Tesi del lavoro

Negli sketch count-distinct distribuiti, hash, seed e parametri non sono dettagli casuali: fanno parte della **semantica** dello sketch e quindi della correttezza del merge.

- Implementazione uniforme di Probabilistic Counting, LogLog, HyperLogLog e HyperLogLog++.
- Framework sperimentale che separa algoritmi, hashing, dataset, metriche e risultati.
- Classificazione operativa dei casi di merge in *valid*, *recoverable* e *invalid*.
- Valutazione empirica di quello che accade quando si ha funzioni di hash e/o parametri differenti, con HLL++ come riferimento pratico.

Delimitazione

Non viene proposto un nuovo stimatore teorico: il contributo è uno studio sistematico del comportamento degli sketch e del merge in condizioni scelte.

$$S = \langle x_1, x_2, \dots, x_s \rangle, \quad x_i \in \mathcal{U}$$

$$f(a) = |\{i \mid x_i = a\}| \quad F_0 = |\{a \in \mathcal{U} \mid f(a) > 0\}|$$

- S è un flusso ordinato di elementi osservati.
- $f(a)$ conta quante volte compare la chiave a .
- F_0 è il numero di elementi distinti nel flusso.
- Nel lavoro si considera il modello *insertion-only*.

Algoritmo di streaming

$$S_i = \langle x_1, \dots, x_i \rangle, \quad M_0 = \text{stato vuoto}$$

$$M_i \leftarrow \text{Update}(M_{i-1}, x_i), \quad \widehat{F}_0(S_i) \leftarrow \text{Query}(M_i)$$

- Lo stato M_i deve essere piccolo rispetto al prefisso processato.
- L'aggiornamento deve essere veloce, perché avviene per ogni elemento.
- L'accuratezza è lo scostamento tra $\widehat{F}_0(S_i)$ e $F_0(S_i)$.
- L'obiettivo è di scambiare la quantità di memoria per una stima controllata.

Stima

Una stima (ε, δ) -approssimata richiede $\Pr(|\widehat{F}_0 - F_0| \leq \varepsilon F_0) \geq 1 - \delta$.

Linea evolutiva degli algoritmi di sketch

Metodo	Stato mantenuto	Miglioramento introdotto
Probabilistic Counting	Una bitmap	-
PCSA	Più bitmap virtuali	Mediazione su sottostrutture
LogLog	Registri	Massimi di rango
HyperLogLog	Registri	Media armonica e correzioni
HyperLogLog++	Registri con strutture sparse/dense	Bias correction e gestione pratica dei regimi piccoli

Struttura della famiglia LogLog/HLL

Precisione k : si usano $m = 2^k$ registri $M[0], \dots, M[m-1]$. Con un hash di L bit:

$$h(x) = \underbrace{j}_{k \text{ bit}} \parallel \underbrace{w}_{L-k \text{ bit}}, \quad r = \text{bin}(j), \quad \rho(w) = 1 + \text{zeri iniziali in } w$$

Aggiornamento

$$M[r] \leftarrow \max\{M[r], \rho(w)\}$$

Ogni registro memorizza il massimo rango osservato sugli elementi assegnati a quel registro.

Stima HyperLogLog

$$Z = \sum_{r=0}^{m-1} 2^{-M[r]}, \quad E = \alpha_m \frac{m^2}{Z}$$

La media armonica riduce il peso dei registri estremi.

LogLog, HyperLogLog e HyperLogLog++ condividono questa struttura a registri. HLL++ aggiunge hash a 64 bit, rappresentazione sparsa/densa e bias correction.

Merge e compatibilità semantica

Obiettivo del merge

Il merge corretto richiede

$$\text{Query}(K_1 \oplus_{\mathcal{A}} K_2) = \text{Query}(K_{\text{serial}})$$

Famiglia bitmap

Probabilistic Counting e PCSA.

$$B = B_1 \vee B_2$$

Merge bit a bit dello stato.

Famiglia a registri

LogLog, HLL e HLL++.

$$\forall j \in \{0, \dots, m-1\}, \quad M[j] = \max\{M_1[j], M_2[j]\}$$

Merge componente per componente.

Compatibilità richiesta

L'operazione $\oplus_{\mathcal{A}}$ è semanticamente corretta solo se restano invariati **algoritmo di sketch**, **semantica hash/seed** e **parametri strutturali dello stato**.

Classificazione dei casi di merge

valid	recoverable	invalid
Merge diretto. Stesso algoritmo, stessa hash, stesso seed, stessi parametri.	Merge corretto dopo normalizzazione. HLL++ con precisioni diverse e stessa semantica hash.	Merge privo di significato nel modello. Hash family o seed diversi.

Strategie operative: direct, reject, reduce_then_merge; unsafe_naive_merge è solo un controllo negativo.

Infrastruttura sperimentale

Algoritmi

Interfaccia comune per process, count, merge, reset.

Implementazioni: PC, LogLog, HLL, HLL++ e riferimento esatto.

Hash e dataset

Funzioni di hash separate dal livello dello sketch.

Seed, parametri e metadati del dataset sono tracciati in modo esplicito.

Valutazione

Modalità streaming, merge omogeneo e merge eterogeneo.

Metriche e risultati vengono serializzati in CSV riproducibili.

Scelta progettuale

Il framework non serve solo a eseguire gli algoritmi, ma a rendere osservabile la compatibilità semantica del merge sotto condizioni controllate e riproducibili.

Dataset controllato e validazione

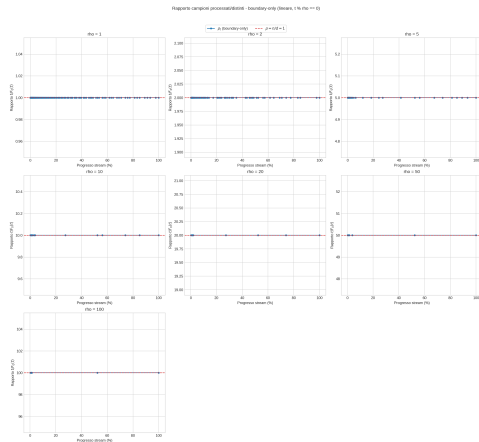
- Dataset sintetico controllato con $n = 10^7$, 50 partizioni, seed = 21041998.
- Rapporto controllato tra elementi processati e distinti:

$$\rho = \frac{n}{d} \in \{100, 50, 20, 10, 5, 2, 1\}.$$

- Una nuova prima occorrenza per ogni blocco di lunghezza ρ :

$$F_0(j \cdot \rho) = j.$$

- Il bitset di verità permette di ricostruire $F_0(t)$ sui prefissi.



La validazione empirica conferma il comportamento atteso di $\rho_t = t/F_0(t)$ ai bordi di blocco.

Test dedicati coprono algoritmi, proprietà del merge, framework, schema CSV e generatore del dataset.

Streaming

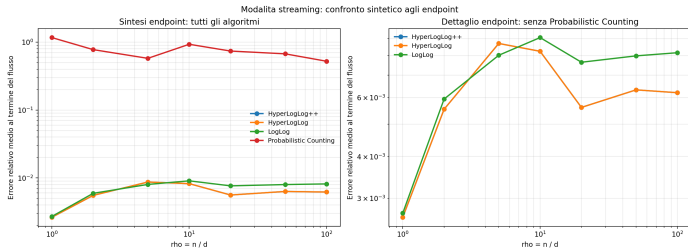
- Due viste: stima in funzione di $F_0(t)$ e stima in funzione di $\rho_t = t/F_0(t)$.
- Sweep dei parametri su $\rho \in \{1, 10, 100\}$.
- HLL++ con $k \in \{8, 10, 12, 14, 16, 18\}$.
- HLL e LogLog con $k \in \{4, 6, 8, 10, 12, 14, 16\}$.
- Probabilistic Counting con $L \in \{4, 8, 12, 16, 20, 24, 28, 31\}$.

Merge HLL++

- Caso omogeneo: riferimento del caso valid.
- Caso eterogeneo con hash mismatch: seed mismatch e family mismatch.
- Caso eterogeneo con precision mismatch: coppie (10, 14), (10, 18), (14, 18) e inverse.
- Strategie: direct, reject, reduce_then_merge, unsafe_naive_merge.

Metriche: stima media, dispersione, errore relativo medio ai punti finali e confronto tra merge e riferimento seriale.

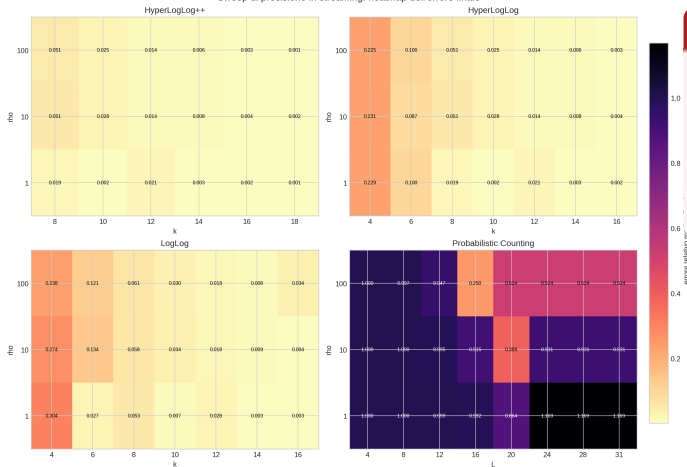
Risultato 1: comportamento degli algoritmi in streaming



- Su $F_0(t)$, HLL e HLL++ sono quasi sovrapposti: errore medio circa 0.00618.
- Su ρ_t , HLL++ è il migliore: $3.95 \cdot 10^{-4}$, contro $2.08 \cdot 10^{-3}$ di HLL.
- LogLog è più sensibile alla vista sperimentale e Probabilistic Counting resta nettamente più instabile.
- Nel dominio osservato, HLL++ è il metodo con il comportamento più regolare.

Risultato 2: l'aumento di precisione favorisce soprattutto HLL++

Sweep di precisione in streaming: heatmap dell'errore finale



Miglior parametro osservato

- HLL++: $k = 18$ per tutti i valori di ρ .
 - HLL: $k = 16$ in media; con $\rho = 1$, $k = 10$.
 - LogLog: $k = 14$ in media; con $\rho = 10$, $k = 16$.
 - PC: $L = 20$ in media; con $\rho = 100$, $L = 16$.
-
- HLL++ è il metodo con risposta più regolare all'aumento di precisione.
 - Negli altri algoritmi il vantaggio del parametro dipende di più dal regime sperimentale.

Risultato 3: il merge omogeneo realizza il caso valid

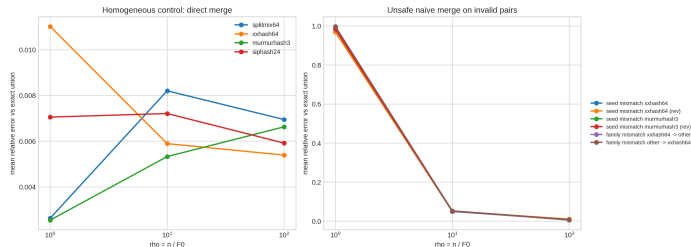
- Configurazione: HyperLogLog++, $\rho \in \{1, 10, 100\}$, $k \in \{10, 14, 18\}$, quattro funzioni di hash.
- Condizione di omogeneità: stessa funzione di hash, stesso seed e stessi parametri sui due lati.
- Totale osservato: 36 configurazioni e 900 coppie di merge.

$$\Delta_{abs} = 0, \quad \Delta_{rel} = 0$$

ρ	d	Coppie totali	Coppie con delta $\neq 0$	Max Δ_{rel}
1	10^7	300	0	0
10	10^6	300	0	0
100	10^5	300	0	0

Nel dominio osservato, il merge omogeneo coincide esattamente con il riferimento seriale e costituisce il controllo sperimentale del caso valid.

Risultato 4: hash mismatch = caso invalid

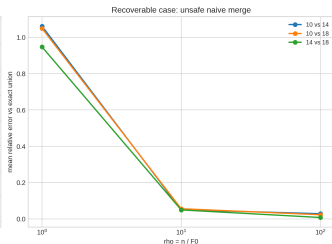
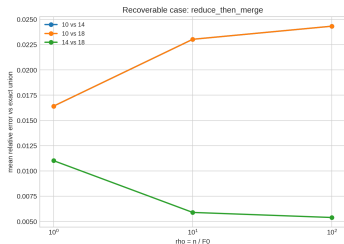


- Seed mismatch: stessa hash xxhash64 o murmurhash3, ma seed diversi.
- Family mismatch: xxhash64 contro splitmix64, murmurhash3 o siphash24; anche direzione inversa.
- Controllo omogeneo locale: errore medio = 0.006233.
- Casi invalid: errore medio tra 0.342374 e 0.352802.
- Errore massimo osservato: 1.009634.
- Strategia corretta: reject; il merge forzato è solo controllo negativo.

Conclusione

Se cambia la semantica dell'hash, il merge forzato perde significato. In questo contesto il caso va rifiutato.

Risultato 5: precision mismatch = caso recoverable



- Hash xxhash64, stesso seed.
- Coppie: (10, 14), (10, 18), (14, 18) e versioni invertite.
- reject: nessuna stima di merge, riferimento seriale = 0.016651.
- La normalizzazione avviene alla precisione minima comune.

Strategia	$\bar{\epsilon}_{merge}$
unsafe_naive_merge	0.363882
reduce_then_merge	0.016651

- reduce_then_merge coincide con il riferimento corretto.
- Il merge forzato raggiunge massimo 1.061378 e degrado medio 32.37.

Sintesi dei risultati e contributi

Risultati empirici

- HLL++ è il metodo più stabile nel dominio osservato.
- Il merge omogeneo coincide col riferimento seriale.
- L'eterogeneità dell'hash rompe la correttezza del merge.
- Il mismatch di precisione è recuperabile solo tramite normalizzazione esplicita.

Contributi del lavoro

- Implementazione uniforme di più algoritmi count-distinct.
- Infrastruttura sperimentale riproducibile per streaming e merge distribuito.
- Evidenza sperimentale sul ruolo della compatibilità semantica nel merge.
- Lessico operativo `valid/recoverable/invalid`.

Take-away

Negli sketch count-distinct distribuiti, hash e parametri fanno parte della semantica dello sketch: senza compatibilità semantica il merge non è un'operazione neutra.

Limiti del lavoro

- Dataset soprattutto sintetico e controllato
- Studio del merge concentrato soprattutto su casi pairwise
- Analisi eterogenea limitata a hash mismatch e precision mismatch
- Parte di merge sviluppata soprattutto su HLL++

Implicazione metodologica

- I risultati chiariscono bene il problema semantico di base, ma non esauriscono la varietà dei flussi reali e delle topologie distribuite possibili.
- Il framework è pensato per estendere l'analisi ad altri assi di eterogeneità e ad altri sketch.

- Portare il framework su dati reali o su sorgenti di flusso come Apache Kafka.
- Studiare topologie di merge più ampie e il punto ottimale in cui ridurre alla precisione minima.
- Estendere i casi eterogenei ad altri assi semanticamente chiari oltre a hash e precisione.
- Integrare sketch count-distinct più recenti e misurare il costo cumulativo della normalizzazione e del merge.

Grazie per l'attenzione.